CAR DEALERSHIP SYSTEM

Main
+ main(args: String[]): void

AppSettings
+ DEFAULT_LOGO_TEXT: String - filePath: String - logoPath: String - logoText: String - showLogoText: boolean - logoImage: BufferedImage - loadedPath: String
+ AppSettings(fileName: String) + load(): void + save(): void + getLogoText(): String + setLogoText(text: String): void + getLogoMain(): String + getLogoAccent(): String + isShowLogoText(): boolean + setShowLogoText(showLogoText: boolean): void + hasCustomLogo(): boolean + getLogoImage(): BufferedImage + setLogoFile(source: File): void + resetLogo(): void

CarRepository
+ CarRepository() # parse(fields: String[]): Car + findByStatus(status: String): List<Car> + chassisExists(chassisNo: String, exceptCarId: String): boolean

CustomerRepository
+ CustomerRepository() # parse(fields: String[]): Customer + search(keyword: String): List<Customer>

DataStore
+ DEFAULT_ADMIN_USERNAME: String + DEFAULT_ADMIN_PASSWORD: String - INSTANCE: DataStore + users: UserRepository + cars: CarRepository + customers: CustomerRepository + orders: OrderRepository + payments: PaymentRepository + handovers: HandoverRepository + logs: LogRepository + settings: AppSettings
- DataStore() + get(): DataStore + initialize(): boolean + log(action: String, details: String): void + customerName(customerId: String): String + carTitle(carId: String): String

HandoverRepository

+ HandoverRepository()
parse(fields: String[]): Handover
+ findByOrder(orderId: String): Handover

LogRepository

+ LogRepository()
parse(fields: String[]): ActivityLog
+ recent(limit: int): List<ActivityLog>

OrderRepository

+ OrderRepository()
parse(fields: String[]): Order
+ findByCustomer(customerId: String): List<Order>
+ findByOrderStatus(status: String): List<Order>
+ carHasOrders(carId: String): boolean

PaymentRepository

+ PaymentRepository()
parse(fields: String[]): Payment
+ findByOrder(orderId: String): List<Payment>

Repository

- filePath: String
- prefix: String
- items: List<T>

Repository(fileName: String, prefix: String)
parse(fields: String[]): T
+ ensureStorage(): void
+ load(): void
- save(): void
+ getAll(): List<T>
+ findById(id: String): T
+ add(item: T): String
+ update(item: T): void
+ delete(id: String): void
- nextId(): String

Session

- currentUser: User

- Session()
+ login(user: User): void
+ logout(): void
+ getUser(): User

UserRepository

+ UserRepository()
parse(fields: String[]): User
+ findByUsername(username: String): User

ActivityLog

- timestamp: String
- username: String
- role: String
- action: String

- details: String
+ ActivityLog() + ActivityLog(timestamp: String, username: String, role: String, action: String, details: String) + getTimestamp(): String + getUsername(): String + getRole(): String + getAction(): String + getDetails(): String + getSummary(): String + toFileLine(): String + fromFields(f: String[]): ActivityLog

Admin
+ Admin(id: String) + getRole(): String + canManageCars(): boolean + canManageStaff(): boolean + canViewReports(): boolean + canChangePrices(): boolean + canApproveOrders(): boolean + canBeBlocked(): boolean

BankTransferPayment
+ BANK_HANDLING_FEE: double
+ getMethodName(): String + requiresVerification(): boolean + calculateCharge(carPrice: double): double + validateReference(reference: String): void

Car
+ AVAILABLE: String + RESERVED: String + UNAVAILABLE: String + RELEASED: String + BRANDS: String[] + BODY_TYPES: String[] + TRANSMISSIONS: String[] + FUEL_TYPES: String[] + CONDITIONS: String[] - brand: String - model: String - variant: String - year: int - bodyType: String - color: String - transmission: String - fuelType: String - engine: String - mileage: int - condition: String - seats: int - chassisNo: String - price: double - status: String - description: String - imagePath: String - dateAdded: String
+ Car() + getBrand(): String + setBrand(brand: String): void + getModel(): String

+ setModel(model: String): void + getVariant(): String + setVariant(variant: String): void + getYear(): int + setYear(year: int): void + getBodyType(): String + setBodyType(bodyType: String): void + getColor(): String + setColor(color: String): void + getTransmission(): String + setTransmission(transmission: String): void + getFuelType(): String + setFuelType(fuelType: String): void + getEngine(): String + setEngine(engine: String): void + getMileage(): int + setMileage(mileage: int): void + getCondition(): String + setCondition(condition: String): void + getSeats(): int + setSeats(seats: int): void + getChassisNo(): String + setChassisNo(chassisNo: String): void + getPrice(): double + setPrice(price: double): void + getStatus(): String + setStatus(status: String): void + getDescription(): String + setDescription(description: String): void + getImagePath(): String + setImagePath(imagePath: String): void + getDateAdded(): String + setDateAdded(dateAdded: String): void + isAvailable(): boolean + getTitle(): String + getSummary(): String + toFileLine(): String + fromFields(f: String[]): Car

CashPayment
+ getMethodName(): String + requiresVerification(): boolean + calculateCharge(carPrice: double): double + validateReference(reference: String): void

Customer
+ ID_TYPES: String[] - address: String - idType: String - idNumber: String - dateRegistered: String
+ Customer() + getAddress(): String + setAddress(address: String): void + getIdType(): String + setIdType(idType: String): void + getIdNumber(): String + setIdNumber(idNumber: String): void + getDateRegistered(): String + setDateRegistered(dateRegistered: String): void + getPersonType(): String

+ getSummary(): String + toFileLine(): String + fromFields(f: String[]): Customer

Entity
- id: String
Entity(id: String) + getId(): String + setId(id: String): void *+ toFileLine(): String* *+ getSummary(): String* + toString(): String

FinancingPayment
+ PROCESSING_RATE: double
+ getMethodName(): String + requiresVerification(): boolean + calculateCharge(carPrice: double): double + validateReference(reference: String): void

Handover
- orderId: String - carId: String - customerId: String - recipientName: String - staffUsername: String - dateReleased: String - notes: String
+ Handover() + getOrderId(): String + setOrderId(orderId: String): void + getCarId(): String + setCarId(carId: String): void + getCustomerId(): String + setCustomerId(customerId: String): void + getRecipientName(): String + setRecipientName(recipientName: String): void + getStaffUsername(): String + setStaffUsername(staffUsername: String): void + getDateReleased(): String + setDateReleased(dateReleased: String): void + getNotes(): String + setNotes(notes: String): void + getSummary(): String + toFileLine(): String + fromFields(f: String[]): Handover

Order
+ DOCUMENTATION_FEE: double + PENDING: String + UNDER_REVIEW: String + AWAITING_PAYMENT: String + READY: String + COMPLETED: String + CANCELLED: String + ON_HOLD: String + UNPAID: String + PAY_PENDING: String + PARTIALLY_PAID: String + PAID: String + FAILED: String

+ REFUNDED: String - customerId: String - carId: String - staffUsername: String - paymentMethod: String - carPrice: double - charges: double - total: double - orderStatus: String - paymentStatus: String - dateCreated: String - notes: String
+ Order() + getCustomerId(): String + setCustomerId(customerId: String): void + getCarId(): String + setCarId(carId: String): void + getStaffUsername(): String + setStaffUsername(staffUsername: String): void + getPaymentMethod(): String + setPaymentMethod(paymentMethod: String): void + getCarPrice(): double + setCarPrice(carPrice: double): void + getCharges(): double + setCharges(charges: double): void + getTotal(): double + setTotal(total: double): void + getOrderStatus(): String + setOrderStatus(orderStatus: String): void + getPaymentStatus(): String + setPaymentStatus(paymentStatus: String): void + getDateCreated(): String + setDateCreated(dateCreated: String): void + getNotes(): String + setNotes(notes: String): void + isActive(): boolean + getSummary(): String + toFileLine(): String + fromFields(f: String[]): Order

Payment
+ CASH: String + BANK: String + FINANCING: String + METHODS: String[] + PENDING_VERIFICATION: String + PAID: String + FAILED: String + REFUNDED: String - orderId: String - amount: double - status: String - reference: String - receiptNo: String - recordedBy: String - dateRecorded: String
Payment() *+ getMethodName(): String* *+ requiresVerification(): boolean* *+ calculateCharge(carPrice: double): double* *+ validateReference(reference: String): void* + forMethod(method: String): Payment + getOrderId(): String

+ setOrderId(orderId: String): void + getAmount(): double + setAmount(amount: double): void + getStatus(): String + setStatus(status: String): void + getReference(): String + setReference(reference: String): void + getReceiptNo(): String + setReceiptNo(receiptNo: String): void + getRecordedBy(): String + setRecordedBy(recordedBy: String): void + getDateRecorded(): String + setDateRecorded(dateRecorded: String): void + getSummary(): String + toFileLine(): String + fromFields(f: String[]): Payment

Person
- fullName: String - contactNo: String - email: String
Person(id: String) + getFullName(): String + setFullName(fullName: String): void + getContactNo(): String + setContactNo(contactNo: String): void + getEmail(): String + setEmail(email: String): void *+ getPersonType(): String*

Staff
+ Staff(id: String) + getRole(): String + canManageCars(): boolean + canManageStaff(): boolean + canViewReports(): boolean + canChangePrices(): boolean + canApproveOrders(): boolean + canBeBlocked(): boolean

User
+ ACTIVE: String + PENDING: String + BLOCKED: String + DEACTIVATED: String - username: String - passwordHash: String - status: String - mustChangePassword: boolean - failedAttempts: int - dateCreated: String
User(id: String) *+ getRole(): String* *+ canManageCars(): boolean* *+ canManageStaff(): boolean* *+ canViewReports(): boolean* *+ canChangePrices(): boolean* *+ canApproveOrders(): boolean* *+ canBeBlocked(): boolean* + getPersonType(): String + getUsername(): String

+ setUsername(username: String): void + getPasswordHash(): String + setPasswordHash(passwordHash: String): void + getStatus(): String + setStatus(status: String): void + isMustChangePassword(): boolean + setMustChangePassword(mustChangePassword: boolean): void + getFailedAttempts(): int + setFailedAttempts(failedAttempts: int): void + getDateCreated(): String + setDateCreated(dateCreated: String): void + isActive(): boolean + getSummary(): String + toFileLine(): String + fromFields(f: String[]): User

AuthException
- terminate: boolean
+ AuthException(message: String, terminate: boolean) + mustTerminate(): boolean

AuthService
+ MAX_TRIES: int - db: DataStore - unknownUserFailures: int
+ login(username: String, password: String): User - failUnknownUser(): User - checkAccountStatus(user: User): void - failWrongPassword(user: User): User + signUp(fullName: String, contact: String, email: String, username: String, password: String, confirm: String): void + createAccount(admin: boolean, fullName: String, contact: String, email: String, username: String): String - buildUser(admin: boolean, fullName: String, contact: String, email: String, username: String): User + updateDetails(user: User, fullName: String, contact: String, email: String): void + setStatus(target: User, newStatus: String): void + resetPassword(target: User): String + deleteAccount(target: User): void + changePassword(user: User, oldPassword: String, newPassword: String, confirm: String): void + confirmAdminPassword(password: String): boolean - requireAdmin(): void

DocumentBuilder
- BAR: String - LINE: String - WIDTH: int
- DocumentBuilder() - row(label: String, value: String): String - center(text: String): String - heading(title: String): String - paragraph(text: String): String - intro(name: String, message: String): String - outro(message: String, preparedBy: String): String - currentUserName(): String - orderBody(o: Order): String + orderSlip(o: Order): String + receipt(p: Payment): String + handoverForm(h: Handover): String + transactionDetails(o: Order): String + customerHistory(c: Customer): String

HandoverService
- db: DataStore

+ release(selected: Order, recipient: String, notes: String, idChecked: boolean, carChecked: boolean): Handover - checkReleaseConditions(order: Order): void

OrderService
- db: DataStore - paymentService: PaymentService
+ calculateCharges(car: Car, method: String): double + createOrder(customer: Customer, selectedCar: Car, method: String, notes: String): Order + approve(selected: Order): void + hold(selected: Order): void + resume(selected: Order): void + cancel(selected: Order): void - fresh(selected: Order): Order - requireApprover(): void

PaymentService
- db: DataStore
+ confirmedAmount(orderId: String): double + pendingAmount(orderId: String): double - sum(orderId: String, status: String): double + amountDue(order: Order): double + record(selected: Order, amount: double, reference: String): Payment - checkOrderCanReceivePayment(order: Order): void + verify(selected: Payment, approve: boolean): boolean + refund(selected: Payment): void - confirm(payment: Payment): void + recompute(order: Order): void

ReportService
+ REPORT_TYPES: String[] + CHART_MODES: String[] - db: DataStore
+ adminStats(): Map<String, String> + staffStats(): Map<String, String> + salesChart(mode: String): Map<String, Double> - slotDate(mode: String, today: LocalDate, back: int): LocalDate - sameSlot(mode: String, date: LocalDate, slot: LocalDate): boolean - chartLabel(mode: String, today: LocalDate, back: int, day: DateTimeFormatter, month: DateTimeFormatter): String + generate(type: String): String - centered(text: String): String - introFor(type: String): String - salesReport(sb: StringBuilder): void - inventoryReport(sb: StringBuilder): void - orderReport(sb: StringBuilder): void - paymentReport(sb: StringBuilder): void - staffReport(sb: StringBuilder): void

ActivityLogPanel
- model: DefaultTableModel - search: JTextField
+ ActivityLogPanel() + refresh(): void

AdminDashboard
+ AdminDashboard(user: User) # buildPages(): void

BaseDashboard

user: User - cards: CardLayout - content: JPanel - nav: JPanel - pageTitle: JLabel - logo: LogoPanel - BasePanel: Map<String, - NavButton: Map<String, - firstPage: String - currentPage: String
BaseDashboard(user: User) *# buildPages(): void* # addPage(key: String, title: String, tooltip: String, panel: BasePanel): void # showFirstPage(): void + showPage(key: String): void - buildTopBar(): JPanel - buildNavStrip(): JPanel - buildFooter(): JPanel - buildSettingsMenu(): JPopupMenu - menuItem(text: String, action: Runnable): JMenuItem - logout(): void - switchAccount(): void

BaseDialog
- title: String - subtitle: String - okText: String - saved: boolean
BaseDialog(owner: Window, title: String, subtitle: String, okText: String) *# buildForm(): JComponent* *# onSubmit(): void* # init(): void - submit(): void + showDialog(): boolean

BasePanel
db: DataStore
BasePanel() *+ refresh(): void* # top(title: String, subtitle: String, tools: JComponent...): JPanel # liveSearch(field: JTextField, action: Runnable): void # owner(): Window

CarFormDialog
- IMG_W: int - IMG_H: int - existing: Car - brand: JComboBox<String> - model: JTextField - variant: JTextField - year: JTextField - bodyType: JComboBox<String> - color: JTextField - transmission: JComboBox<String> - fuel: JComboBox<String> - engine: JTextField - mileage: JTextField - condition: JComboBox<String> - seats: JTextField - chassis: JTextField - price: JTextField - status: JComboBox<String>

- description: JTextArea - preview: JLabel - imagePath: String - pendingImage: File
+ CarFormDialog(owner: Window, existing: Car) - loadExisting(c: Car): void # buildForm(): JComponent - chooseImage(): void # onSubmit(): void

CarManagementPanel
- model: DefaultTableModel - table: JTable - search: JTextField - statusFilter: JComboBox<String> - shown: List<Car>
+ CarManagementPanel() + refresh(): void - selected(): Car - editSelected(): void - archiveSelected(): void - deleteSelected(): void

ChangePasswordDialog
- user: User - auth: AuthService - current: JPasswordField - newPass: JPasswordField - confirm: JPasswordField
+ ChangePasswordDialog(owner: Window, user: User, forced: boolean) # buildForm(): JComponent # onSubmit(): void

CustomerFormDialog
- existing: Customer - fullName: JTextField - contact: JTextField - email: JTextField - address: JTextField - idType: JComboBox<String> - idNumber: JTextField
+ CustomerFormDialog(owner: Window, existing: Customer) # buildForm(): JComponent # onSubmit(): void + fillFrom(c: Customer, name: String, phone: String, mail: String, addr: String, idKind: String, idNo: String): void

CustomersPanel
- admin: boolean - model: DefaultTableModel - table: JTable - search: JTextField - shown: List<Customer>
+ CustomersPanel(admin: boolean) + refresh(): void - selected(): Customer - editSelected(): void - deleteSelected(): void

Dialogs
- Dialogs()

+ info(parent: Component, message: String): void + success(parent: Component, message: String): void + warn(parent: Component, message: String): void + warn(parent: Component, title: String, message: String): void + error(parent: Component, message: String): void + error(parent: Component, title: String, message: String): void + confirm(parent: Component, message: String): boolean + confirmAdminPassword(parent: Component, action: String): boolean

HandoverDialog
- order: Order - service: HandoverService - paymentOk: JCheckBox - idOk: JCheckBox - carOk: JCheckBox - recipient: JTextField - notes: JTextArea - created: Handover
+ HandoverDialog(owner: Window, order: Order) # buildForm(): JComponent # onSubmit(): void + getCreatedHandover(): Handover

HandoverPanel
- readyModel: DefaultTableModel - readyTable: JTable - doneModel: DefaultTableModel - doneTable: JTable - ready: List<Order> - done: List<Handover>
+ HandoverPanel() - section(title: String, table: JTable): JPanel + refresh(): void - releaseSelected(): void

HomePanel
- ACCENTS: Color[] - admin: boolean - reports: ReportService - cards: JPanel - subtitle: JLabel - model: DefaultTableModel - tableTitle: JLabel - chart: SalesChartPanel - chartMode: JComboBox<String> - chartTotal: JLabel
+ HomePanel(admin: boolean) - buildAdminCenter(tableCard: JPanel): JPanel - refreshChart(): void + refresh(): void - statCard(label: String, value: String, accent: Color): JPanel

LoginFrame
- auth: AuthService - username: JTextField - password: JPasswordField - show: JCheckBox
+ LoginFrame() - buildForm(): JPanel - left(c: javax.swing.JComponent): JPanel - full(c: javax.swing.JComponent): JPanel

- doLogin(): void - openDashboard(user: User): void

LogoDialog
- settings: AppSettings - text: JTextField - showText: JCheckBox - fileName: JLabel - previewHolder: JPanel - chosenFile: File - previewImage: BufferedImage - useDefault: boolean
+ LogoDialog(owner: Window) # buildForm(): JComponent - choosePicture(): void - updatePreview(): void # onSubmit(): void

LogoPanel
- height: int - textColor: Color - font: Font - live: boolean - image: BufferedImage - main: String - accent: String - showText: boolean
+ LogoPanel(height: int, textColor: Color) + LogoPanel(height: int, textColor: Color, image: BufferedImage, text: String, showText: boolean) - LogoPanel(height: int, textColor: Color, image: BufferedImage, text: String, showText: boolean, live: boolean) + reload(): void - show(picture: BufferedImage, text: String, visible: boolean): void - imageWidth(): int - resize(): void # paintComponent(g: Graphics): void + isLive(): boolean

NewOrderPanel
- orderService: OrderService - onSaved: Runnable - updating: boolean - searchCustomer: JTextField - customerBox: JComboBox<String> - matches: List<Customer> - name: JTextField - phone: JTextField - email: JTextField - address: JTextField - idType: JComboBox<String> - idNo: JTextField - carBox: JComboBox<String> - cars: List<Car> - carImage: JLabel - carInfo: JLabel - methodBox: JComboBox<String> - notes: JTextArea - priceLbl: JLabel - feeLbl: JLabel - chargeLbl: JLabel - totalLbl: JLabel
+ NewOrderPanel(onSaved: Runnable) - customerCard(): JPanel

- vehicleCard(): JPanel - paymentCard(): JPanel - addSummaryRow(panel: JPanel, row: int, label: String, value: JLabel): void + refresh(): void + setCar(car: Car): void - loadCars(selectId: String): void - selectedCar(): Car - carChosen(): void - updateSummary(): void - findCustomers(): void - customerChosen(): void - buildCustomer(): Customer - saveOrder(): void - clearForm(): void

OrdersPanel
- admin: boolean - service: OrderService - model: DefaultTableModel - table: JTable - search: JTextField - statusFilter: JComboBox<String> - shown: List<Order>
+ OrdersPanel(admin: boolean) + refresh(): void - selected(): Order - act(action: OrderAction, verb: String): void - cancelSelected(): void

PaymentDialog
- db: DataStore - service: PaymentService - orders: List<Order> - orderBox: JComboBox<String> - customer: JLabel - method: JLabel - total: JLabel - paid: JLabel - due: JLabel - amount: JTextField - reference: JTextField - created: Payment
+ PaymentDialog(owner: Window, preselected: Order) - selected(): Order - showOrder(): void # buildForm(): JComponent # onSubmit(): void + getCreatedPayment(): Payment

PaymentsPanel
- admin: boolean - service: PaymentService - model: DefaultTableModel - table: JTable - search: JTextField - statusFilter: JComboBox<String> - shown: List<Payment>
+ PaymentsPanel(admin: boolean) + refresh(): void - selected(): Payment - recordPayment(): void - verifySelected(approve: boolean): void

- refundSelected(): void - showReceipt(p: Payment): void

ReportsPanel
- reports: ReportService - type: JComboBox<String> - output: JTextArea
+ ReportsPanel() - generate(): void + refresh(): void

SalesChartPanel
- barColor: Color - Double: Map<String, - barShapes: List<RoundRectangle2D> - hovered: int
+ SalesChartPanel(barColor: Color) + setData(values: Map<String, Double>): void - compact(amount: double): String - trim(value: double): String # paintComponent(g: Graphics): void

SignUpDialog
- auth: AuthService - fullName: JTextField - contact: JTextField - email: JTextField - username: JTextField - password: JPasswordField - confirm: JPasswordField - strength: JLabel - show: JCheckBox
+ SignUpDialog(owner: Window) # buildForm(): JComponent # onSubmit(): void

SlipDialog
+ SlipDialog(owner: Window, title: String, text: String, fileName: String)

StaffDashboard
- orderPanel: NewOrderPanel
+ StaffDashboard(user: User) # buildPages(): void

StaffFormDialog
- existing: User - auth: AuthService - role: JComboBox<String> - fullName: JTextField - username: JTextField - contact: JTextField - email: JTextField
+ StaffFormDialog(owner: Window, existing: User) # buildForm(): JComponent # onSubmit(): void

StaffManagementPanel
- auth: AuthService - model: DefaultTableModel

- table: JTable - search: JTextField - shown: List<User>
+ StaffManagementPanel() + refresh(): void - selected(): User - changeStatus(requiredCurrent: String, newStatus: String, errorMessage: String): void - toggleSelected(): void - resetSelected(): void - deleteSelected(): void

Theme
+ APP_NAME: String + BLACK: Color + CHARCOAL: Color + RED: Color + DARK_RED: Color + BG: Color + CARD: Color + TEXT: Color + MUTED: Color + BORDER: Color + SOFT_RED: Color + SUCCESS: Color + WARNING: Color + INFO: Color + GRAY: Color + FONT: Font + BOLD: Font + TITLE: Font + HEADING: Font + SMALL: Font + MONO: Font
- Theme() + install(): void + button(text: String, color: Color): JButton + primary(text: String): JButton + secondary(text: String): JButton + neutral(text: String): JButton + label(text: String): JLabel + muted(text: String): JLabel + title(text: String): JLabel + heading(text: String): JLabel + textField(columns: int): JTextField + passwordField(columns: int): JPasswordField - styleField(f: JTextField): void - fieldBorder(line: Color): javax.swing.border.Border + combo(items: String...): JComboBox<String> + textArea(rows: int, columns: int): JTextArea + scroll(c: Component): JScrollPane + card(): JPanel + addField(form: JPanel, row: int, col: int, label: String, field: JComponent): void + addWide(form: JPanel, row: int, label: String, field: JComponent): void + formPanel(): JPanel + readOnlyModel(columns: String...): DefaultTableModel + table(model: TableModel): JTable + widths(table: JTable, widths: int...): void + selectedModelRow(table: JTable): int + statusColor(text: String): Color

VehicleBrowsePanel
- model: DefaultTableModel

- table: JTable - search: JTextField - bodyFilter: JComboBox<String> - shown: List<Car> - image: JLabel - details: JEditorPane - startOrder: JButton - onStartOrder: Consumer<Car>
+ VehicleBrowsePanel(onStartOrder: Consumer<Car>) + refresh(): void - showDetails(): void

DataException
+ DataException(message: String) + DataException(message: String, cause: Throwable)

ErrorHandler
- ErrorHandler() + execute(parent: Component, action: Action): boolean + logError(error: Throwable): void

FileHandler
+ DATA_DIR: String + IMAGE_DIR: String + DOC_DIR: String
- FileHandler() + path(fileName: String): String + ensureFile(path: String): void + readLines(path: String): List<String> + writeLines(path: String, lines: List<String>): void + appendLine(path: String, line: String): void + writeText(path: String, text: String): void + join(fields: Object...): String + split(line: String): String[] - escape(text: String): String - unescape(text: String): String

Format
- MONEY: DecimalFormat - DATE_TIME: DateTimeFormatter
- Format() + money(amount: double): String + now(): String + today(): String + currentYear(): int + pad(text: String, width: int): String

ImageUtil
- ImageIcon: Map<String,
- ImageUtil() + importImage(source: File): String + importImage(source: File, prefix: String): String + load(path: String, w: int, h: int): ImageIcon - drawPlaceholder(g: Graphics2D, w: int, h: int): void + drawCar(g: Graphics2D, x: int, y: int, w: int, h: int, body: Color, detail: Color): void

PasswordUtil
+ PREFIX: String

- RANDOM: SecureRandom
- PasswordUtil() + generate(): String + hash(username: String, password: String): String + matches(username: String, password: String, storedHash: String): boolean

ValidationException
+ ValidationException(message: String)

Validator
- Validator() + requireText(value: String, field: String): String + requireText(value: String, field: String, min: int, max: int): String + optionalText(value: String, field: String, max: int): String + requireName(value: String, field: String): String + parseMoney(value: String, field: String): double + parseInt(value: String, field: String, min: int, max: int): int + requirePhone(value: String): String + optionalEmail(value: String): String + requireUsername(value: String): String + checkPasswordStrength(password: String): void + passwordStrength(password: String): String + requireSelected(value: Object, field: String): void